

When You Come to a Fork in the Road, Ask Claude

Nick Wanninger

Northwestern University
Evanston, Illinois, United States

Alex Butler

Northwestern University
Evanston, Illinois, United States

Tommy McMichen

Northwestern University
Evanston, Illinois, United States

Abstract

As Moore’s law comes back from a brief hiatus, we need to find a way to waste transistors again. We argue that the most effective use case is to once again beat an old dead horse: branch prediction. Modern chip-multiprocessors (CMPs) leverage branch predictors inspired by the human brain such as perceptrons and neural networks. Orthogonally, natural-language processing has leveraged large language models (LLMs) to tremendous effect, enabling a revolution in the arts, sciences and cultural disruption.

To effectively hitch ourselves to this fast-moving bandwagon, we propose DEEPBRANCH: a novel system that uses LLMs to predict branches in an out-of-order core. While traditional perceptron branch predictors can only model linearly separable branching patterns, DEEPBRANCH is able to model any branching pattern that can be expressed in a natural language. This design enables a branch predictor that is both reprogrammable and modular by changing the system prompt and underlying model. We perform an in-depth evaluation of DEEPBRANCH using a plethora of state-of-the-art large language models, and find that many achieve accuracies on par with their perceptron siblings with significant speedups of up to 1.03× on a high-impact, industry-standard benchmark.

1 Introduction

Branch prediction is hard. In state of the art solutions today, processors are inspired by the mechanical workings of the human brain¹, using perceptrons and neural networks to improve accuracy in a tight silicon area budget [1, 2, 5–12, 15–24, 27, 29]. While it may seem that all the low-hanging fruit has been plucked from the proverbial tree of branch prediction solutions, this paper leverages the modern AI era² to achieve the next generation³ of branch prediction accuracy.

In this paper, we leverage the advent of modern large-language models (LLMs) to provide the next generation of branch predictions: DEEPBRANCH. In DEEPBRANCH, we replace the branch predictor of a conventional out-of-order machine with a prompt to your favorite AI model. Rather than taking an offline approach for hardware prediction mechanisms [28], we use an *online* approach to allow our predictor to better adapt to rapidly changing runtime environments⁴. DEEPBRANCH is able to leverage the strong points of conventional branch predictors⁵ and put their own stochastic sauce on it at times. In addition to exceptional accuracy, DEEPBRANCH

provides detailed insights for branch prediction performance by simply asking it “How are you doing?”. This places it in a space far superior to both conventional branch predictors (which are awkwardly silent) and your own friends (who will start talking about their partner, or the fact that their dog died recently).

We implement DEEPBRANCH in Gem5 [3] to provide vague, cycle-inaccurate performance results⁶. We evaluate a bevy of state-of-the-art models, with and without thinking (§4). To understand the feasibility of DEEPBRANCH in production, we perform back-of-the-napkin calculations for the silicon footprint of our solution and find that DEEPBRANCH occupies an interesting point in the accuracy-space tradeoff and is solely limited by the speed of light [4] (§3). We then half-heartedly admit a few limitations and threats to validity (§5) before overstating our findings in the conclusion (§7).

This paper makes the following contributions:

- Propose DEEPBRANCH, an architecture-AI co-design that leverages modern LLMs to provide accurate and insightful branch predictions (§2).
- Estimates the silicon footprint of implementing DEEPBRANCH, including power and real estate costs (§3).
- Evaluates DEEPBRANCH using a wide array of state-of-the-art large language models to understand the best IPO in which to invest (§4).
- Identifies novel threat models made possible with the attack surface provided by DEEPBRANCH (§5).

2 System Design

DEEPBRANCH replaces the branch predictor in a typical out-of-order machine, as shown in Figure 1. When the frontend encounters a branch instruction, it invokes the LLM with the system prompt:

“You are a hardware branch predictor. Given a branch PC and recent branch history, output exactly one word: TAKEN or NOTTAKEN. No explanation.”

The model is also provided with historical context. The branch history is stored in a PC-indexed hashtable, containing the last 13 branch results for all hashed PCs. The prompt includes the current PC and the corresponding branch results as a sequence of “TAKEN” and “NOTTAKEN” tokens. An example of this is shown below:

Branch history of pc=0x415b18:
NOTTAKEN NOTTAKEN NOTTAKEN NOT-
TAKEN NOTTAKEN NOTTAKEN NOTTAKEN
NOTTAKEN NOTTAKEN NOTTAKEN NOT-
TAKEN NOTTAKEN TAKEN
Branch?

The model response is then parsed to obtain the prediction result with perl-style regular expressions. Note that, even though we asked

¹Out of jealousy, most likely.

²Where we are allowed to freely rediscover old solutions, if they adequately appease the markets that be.

³Which is just like the last generation, but with more buzzwords.

⁴As seen with Claude Code pushing stuff to prod faster than ever.

⁵Since LLMs are the only entities to have read the Patterson and Hennessy book in full, see the bold text in §A.

⁶We leverage Gem5 because the alternative, gate-level simulation, is non-vague, but also cycle-inaccurate, which we argue is much worse.

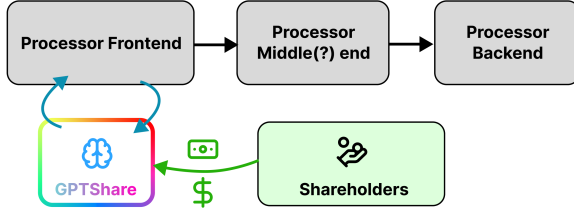


Figure 1: All questions the CPU frontend has are directed to the AI chatbot in DEEPBRANCH, and shareholders are happy to help!

kindly, the model does not always respond with the requested structure, and tends to ramble like Waylon Jennings after he's had a few too many. To be robust to such unstructured outputs and hallucinations, the hardware implements regular expression matching. The prediction is then used by the frontend to issue speculative instructions into the processor pipeline.

With modern thinking models, DEEPBRANCH is able to retrieve fine-grained performance insights from reasoning trajectories. We believe that this new performance analysis strategy marks a fundamental shift. While today's performance engineers need to use tools like *perf* to inspect performance counters from the CPU, DEEPBRANCH enables them to directly query the branch predictor to quickly identify the root cause of performance regressions and identify potential optimizations in the source code. We are already seeing promising approaches to natural language for performance tracing and debugging [14], but we propose taking this to its logical conclusion. We present examples of this novel workflow in Table 1.

3 The Hardware

To make this viable as a branch predictor—which mandates subcycle latency to operate within a single pipeline stage—DEEPBRANCH implements the entire model as a purely combinational circuit. No clock, no SRAM, no sequential logic. Every weight and every arithmetic operation of a single forward pass is physically instantiated as combinational gates. The branch history enters as an input vector in plain english, propagates through the circuit, and the taken/not-taken prediction appears after one propagation delay.

Instead of buying Cadence, we asked Claude Opus 4.6⁷ how this could work and it thought for a while. We aren't sure if this is right (it smells okay), but this is what it came up with:

3.1 The Model Layout

The key trick here is to use **ternary weights**[26] ($w \in \{-1, 0, +1\}$) for the model. This reduces the FFN multiply-accumulate unit to a mux (negate, zero, or pass) plus an adder tree. This uses roughly 100× *less* transistors than a full FP16 multiplier. Attention [25], however, computes QK^T and AV from runtime values and still needs a ton of transistors.

We envision a hardware instantiation of the DEEPBRANCH predictor to use an 8B parameter transformer, with the architecture specified in Table 2.

⁷In the spirit of this paper

At this process node, the transistor density ρ is 100×10^6 transistors per mm^2 . The wafer cost c_w is \$14,000. The supply voltage V_{DD} is 0.85 V, and the gate delay t_g is 15 ps. All values correspond to a 5 nm process node.

Two derived architectural quantities are immediate:

$$d_{\text{head}} = \frac{d_{\text{model}}}{N_{\text{heads}}} = \frac{4096}{32} = 128 \quad (1)$$

$$N_{\text{global}} = \max(1, \text{round}(N_{\text{layers}} \cdot r_g)) \quad (2)$$

$$= \max(1, \text{round}(32 \times \frac{1}{6})) \quad (3)$$

$$= \text{round}(5.333) = 5 \quad (4)$$

$$N_{\text{local}} = N_{\text{layers}} - N_{\text{global}} = 32 - 5 = 27 \quad (5)$$

Five of the 32 layers use full-context (global) attention over the entire 4096-entry branch history. The remaining 27 layers use a sliding window of 1024 entries, attending only to recent history. This follows the Gemma-3 convention of roughly one global attention layer for every five local layers.

FFN Transistors. Each ternary weight needs a 2-bit mux (about 6 transistors to select between $-x$, 0, and $+x$) plus its share of the reduction adder tree (about 4 transistors). We use $\tau_{\text{ternary}} = 10$ transistors per weight:

$$T_{\text{FFN}} = P \times \tau_{\text{ternary}} = \boxed{80 \text{ B transistors}} \quad (6)$$

Global Attention Transistors. Each global attention layer must compute QK^T and AV over the full context. For a single product, the matrix multiply $(S \times d_{\text{head}}) \cdot (d_{\text{head}} \times S)$ requires $S \times d_{\text{head}} \times S$ scalar FP16 multiplications. The factor of 2 accounts for both QK^T and AV , and we have N_{kv} independent KV-head computations across N_{global} layers.

The number of FP16 multipliers is:

$$\begin{aligned} M_{\text{global}} &= S \times d_{\text{head}} \times S \times N_{\text{kv}} \times N_{\text{global}} \times 2 \\ &= 4096 \times 128 \times 4096 \times 32 \times 5 \times 2 \\ &= \boxed{687,194,767,360} \end{aligned} \quad (7)$$

Each FP16 multiplier costs $\tau_{\text{fp16}} = 1000$ transistors (covering the 11-bit mantissa array, exponent adder, normalization/rounding, and sign logic):

$$T_{\text{attn,global}} = M_{\text{global}} \times 1000 \quad (8)$$

$$= 687,194,767,360,000 \quad (9)$$

$$\approx \boxed{687.2 \text{ T transistors}} \quad (10)$$

$$(11)$$

Local Attention Transistors. The same computation, but with the window size $W_{\text{local}} = 1024$ replacing the full context length, across $N_{\text{local}} = 27$ layers:

$$\begin{aligned} M_{\text{local}} &= W_{\text{local}} \times d_{\text{head}} \times W_{\text{local}} \times N_{\text{kv}} \times N_{\text{local}} \times 2 \\ &= 1024 \times 128 \times 1024 \times 32 \times 27 \times 2 \\ &= 231,928,233,984 \end{aligned} \quad (12)$$

Query	Response
“Is my code cache friendly?”	“Great Question! Not particularly. Have you considered restructuring your data layout?”
“How many branch mispredictions occurred?”	“Quite a few, honestly. The loop in <code>malloc.c</code> is especially unpredictable.”
“What is my IPC?”	“Certainly! Here’s a recipe for my Grandma’s Chicken Pot-roast!”
“Where is my hottest code path?”	“Physically or computationally?”

Table 1: The future of perf. We predict improvement as we approach what vibe coders (hereinafter VC’s) and venture capitalists (hereinafter VC’s) refer to as ‘AGI.’

Parameter	Value
Total parameters P	8,000,000,000
Hidden dimension d_{model}	4096
Layers N_{layers}	32
Attention heads N_{heads}	32
KV heads N_{kv}	32
Context length S	4096
Global attention ratio r_g	1/6
Local attention window W_{local}	1024
Process node	TSMC 5N

Table 2: All the fiddly details about the DEEPBRANCH architecture and hardware implementation that reviewers thought were super necessary to have in the paper.

$$T_{\text{attn,local}} = M_{\text{local}} \times 1000 \quad (13)$$

$$= 231,928,233,984,000 \quad (14)$$

$$\approx \boxed{231.9 \text{ T transistors}} \quad (15)$$

Total Logical Transistors.

$$\begin{aligned} T_{\text{logical}} &= T_{\text{FFN}} + T_{\text{attn,global}} + T_{\text{attn,local}} \\ &= 80 \times 10^9 + 687.2 \times 10^{12} + 231.9 \times 10^{12} \\ &= 919,203,001,344,000 \\ &\approx \boxed{919.2 \text{ T transistors}} \end{aligned} \quad (16)$$

Note the dramatic imbalance: the FFN contributes just 80 B transistors (0.009% of the total) while attention accounts for 919.1 T. This is because ternary MACs cost 10 transistors each, while FP16 multipliers cost 1000—and attention must instantiate one multiplier for every element of the $S \times S$ score matrix, for every head and every layer.

Applying Redundancy. At these die sizes, manufacturing defects are inevitable. We apply an arbitrary redundancy factor $R = 10$ to account for spare logic, wire routing, multi-core, error-correcting voter circuits, and yield overhead:

$$T_{\text{total}} = T_{\text{logical}} \times R = 919.2 \times 10^{12} \times 10 = \boxed{9,192 \text{ T transistors}} \quad (17)$$

3.2 Die Area

Dividing the total transistor count by the 5 nm density TSMC claims:

$$A = \frac{T_{\text{total}}}{\rho} = \frac{9,192 \times 10^{12}}{100 \times 10^6 \text{ tr/mm}^2} = 91,920,300 \text{ mm}^2 \approx \boxed{91.9 \text{ m}^2} \quad (18)$$

For reference, this is a square with side length:

$$\sqrt{A} = \sqrt{91,920,300} \approx 9,588 \text{ mm} \approx \boxed{9.59 \text{ m}} \quad (19)$$

For Americans, that is:

$$\frac{9.59 \text{ m}}{3.28084 \text{ ft/m}} = \boxed{31.46325 \text{ ft}} \quad (20)$$

A single 300 mm wafer has only $\sim 60,000 \text{ mm}^2$ of usable area, so this design would require over 1,500 wafers worth of silicon stitched together via wafer-scale integration or an enormous interposer. We leave solving this particular manufacturing conundrum to the smart cookies at TSMC, and await their marketing announcement. The size of this chip is shown next to common household items for scale in Figure 2.

We *predict* this hardware would benefit from being a chiplet architecture to reduce yield troubles. If Apple can glue processors together to make the “Ultra” chips, this should be easy. After all, wafer-scale chiplet architectures really get the investors going!

3.3 Propagation Delay

Since there is no clock, the “speed” of the chip is determined entirely by the worst-case propagation delay from input to output. The critical path passes through all 32 layers in series. Within each layer, three logic stages contribute to the delay:

- (1) **FFN adder tree:** $D_{\text{adder}} = 12$ gate stages (since $\log_2(4096) = 12$ for a d_{model} -wide reduction).
- (2) **Attention sequence reduction:** $\log_2(S) = \log_2(4096) = 12$ gate stages.
- (3) **FP16 multiplier:** $D_{\text{fp16}} = 30$ gate stages for one complete multiply.

Converting the gate delay to nanoseconds:

$$t_g = 15 \text{ ps} = 0.015 \text{ ns} \quad (21)$$

The per-layer delay is:

$$\begin{aligned} t_{\text{layer}} &= (D_{\text{adder}} + \log_2(S) + D_{\text{fp16}}) \times t_g \\ &= (12 + 12 + 30) \times 0.015 \\ &= 54 \times 0.015 = \boxed{0.81 \text{ ns}} \end{aligned} \quad (22)$$

Over all 32 layers:

$$t_{\text{prop}} = N_{\text{layers}} \times t_{\text{layer}} = 32 \times 0.81 = \boxed{25.92 \text{ ns}} \quad (23)$$

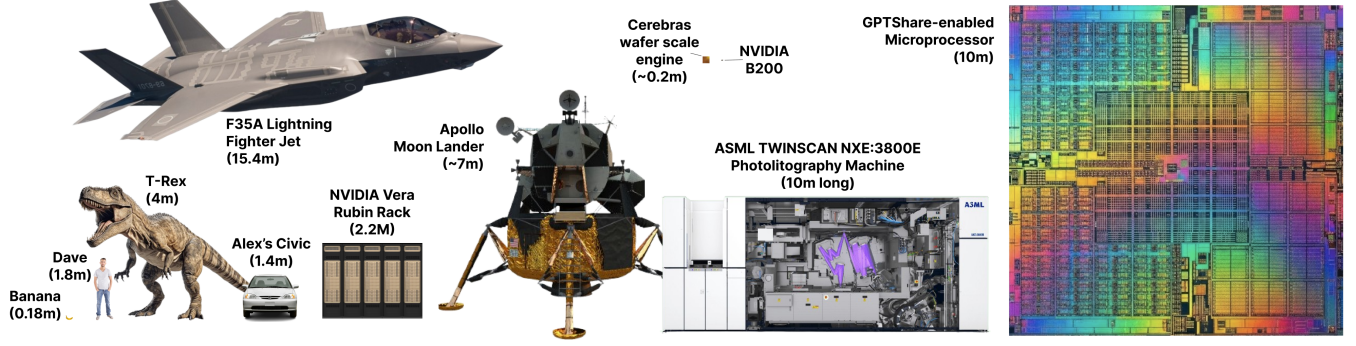


Figure 2: DEEPBRANCH shown to-scale with common household items. Of important note, a DEEPBRANCH enabled microprocessor is only about twice the size of the photolithography machine which creates it.

The maximum prediction rate, feeding a new branch query as soon as the previous prediction stabilizes:

$$f_{\text{pred}} = \frac{1}{t_{\text{prop}}} = \frac{1}{25.92 \times 10^{-9}} \quad (24)$$

$$= \boxed{38,580,247 \text{ predictions/s}} \quad (25)$$

Which is basically like 38MHz.

3.4 Power

Dynamic power in CMOS follows $P = \alpha \cdot C \cdot V_{DD}^2 \cdot f$. We compute energy per token first, using three constants:

Constant	Value	Meaning
C_{tr}	0.5 fF	Capacitance per transistor
α	0.1	Activity factor (fraction switching per token)
V_{DD}	0.85 V	5 nm supply voltage

Note that the energy computation uses T_{logical} (not T_{total}), because redundant/spare transistors do not switch during normal operation:

$$\begin{aligned} E_{\text{token}} &= T_{\text{logical}} \times C_{\text{tr}} \times V_{DD}^2 \times \alpha \\ &= 919.2 \times 10^{12} \times 0.5 \times 10^{-15} \text{ F} \times (0.85)^2 \times 0.1 \\ &= 919.2 \times 10^{12} \times 0.5 \times 10^{-15} \times 0.7225 \times 0.1 \\ &= 919.2 \times 10^{12} \times 3.6125 \times 10^{-17} \\ &= 3.321 \times 10^{-2} \text{ J} \\ &= \boxed{33.21 \text{ mJ/token}} \end{aligned} \quad (26)$$

Peak power at the maximum token rate:

$$\begin{aligned} P_{\text{peak}} &= E_{\text{token}} \times f_{\text{tok}} \\ &= 0.03321 \text{ J} \times 38,580,247 \text{ s}^{-1} \\ &= 1,281,104 \text{ W} \\ &= \boxed{\approx 1.28 \text{ MW}} \end{aligned} \quad (27)$$

3.5 Cost

The total cost has four components.

Silicon. First, the usable area of a 300 mm wafer (85% utilization):

$$A_{\text{wafer}} = \pi \times 150^2 \times 0.85 = 60,083 \text{ mm}^2 \quad (28)$$

The number of wafers needed includes another factor of $R = 10$ for yield loss at these extreme areas:

$$N_w = \frac{A}{A_{\text{wafer}}} \times R = \frac{91,920,300}{60,083} \times 10 = 1,529.9 \times 10 = 15,299 \quad (29)$$

$$C_{\text{Si}} = N_w \times c_w = 15,299 \times \$14,000 = \boxed{\$214.2\text{M}} \quad (30)$$

Packaging. At $\$0.50/\text{mm}^2$ for exotic large-area packaging (custom substrate, silicon interposer, advanced fan-out):

$$C_{\text{pkg}} = A \times \$0.50 = 91,920,300 \times 0.50 = \boxed{\$46.0\text{M}} \quad (31)$$

Non-Recurring Engineering. NRE covers mask sets, EDA licenses, physical design, and verification. A $\$50\text{M}$ base cost plus $\$8/\text{mm}^2$ for complexity-driven scaling:

$$\begin{aligned} C_{\text{NRE}} &= \$50\text{M} + A \times \$8.00 \\ &= \$50,000,000 + 91,920,300 \times 8.00 \\ &= \$50,000,000 + \$735,362,400 \\ &= \boxed{\$785.4\text{M}} \end{aligned} \quad (32)$$

NRE dominates the total cost. This makes sense: designing, verifying, and taping out a circuit of this complexity—even if much of it is regular and repetitive—would be an unprecedented engineering effort.

Infrastructure. Power delivery ($\$5/\text{W}$ at current oil prices) and direct liquid cooling ($\$2/\text{W}$):

$$C_{\text{infra}} = P_{\text{peak}} \times \$7.00 = 1,281,104 \times 7.00 = \boxed{\$9.0\text{M}} \quad (33)$$

Total Cost.

$$\begin{aligned} C_{\text{total}} &= C_{\text{Si}} + C_{\text{pkg}} + C_{\text{NRE}} + C_{\text{infra}} \\ &= \$214.2\text{M} + \$46.0\text{M} + \$785.4\text{M} + \$9.0\text{M} \\ &= \boxed{\$1.054 \text{ B}} \end{aligned} \quad (34)$$

Interestingly, this die is larger than the average studio apartment in Palo Alto, and *still* costs less.


```

struct node *app_x(struct node *curr) {
    struct node *prev = NULL;
    while (curr) {
        struct node *next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return curr;
}

```

Figure 3: APP-X: An important, industry impactful, workload. Our industry contacts at Company A, Company B, and Company C say that this will have a trillion dollar impact on the global economy, far exceeding menial inventions like the printing press.

4 Evaluation

We implement DEEPBRANCH in the Gem5 architectural simulator using a bog-standard O3 x86 core. As lowly grad students who can't afford an apartment with enough floorspace to house such a chip, Gem5 was unfortunately the only option. Our implementation in Gem5 assumes that all of these requests have subcycle latency, as is expected of a branch predictor. We do not model this latency in our results.

We evaluate the predictor on a wide range of industry applications, but only present an application we call APP-X for brevity. APP-X is provided in its entirety in Figure 3.

This application features interesting behavior such as: a while loop. We run the APP-X application many times, sweeping over any of open-weight predictors LLM models we could get our hands on⁸. As an additional axis of evaluation, we allow two of the models (GPT-OSS and Qwen3) to reason/think about the problem before responding. We also include a few off-the-shelf predictors such as “Always Taken”, “Always Nottaken”, and Gem5’s own LTAGE and MultiperspectivePerceptron predictors as baselines representing the boring state-of-the-art.

Figure 4 shows the raw branch prediction accuracy of the various models. Almost all of the models achieve greater than 80% accuracy in the benchmark, lending credence to the approach proposed in this paper. There doesn’t seem to be a high correlation between the parameter count of the model and raw prediction accuracy, and in fact Google’s GEMMA models actually get *worse* accuracy as the model grows. Deepseek, interestingly, was the worst model. It outright *refused* to be a branch predictor, often deflecting to more classical chatbot activities like providing cooking recipes or writing python code. Below is an excerpt from a Deepseek-R1 prediction:

As an AI, I don’t have the ability to predict hardware branch predictions or determine if a branch is TAKEN or NOTTAKEN.

We believe this can be solved with fine-tuning using LoRA, or better prompting.

⁸We thought about using the Claude API, but with the war going on and gas prices what they are, money is rather tight.

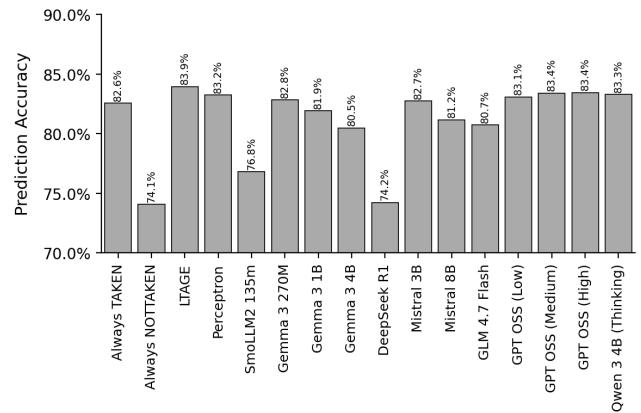


Figure 4: Raw prediction accuracy is pretty much a wash.

The “SmoLLM2” predictor would often lose track of the task at hand, and start ranting about banks in Boston⁹ for thousands of tokens. Sometimes, it would write code which is ostensibly valid perl [13].

While prediction accuracy is great and all, the real meat-and-potatoes of any hardware change is end-to-end speedup, shown in Figure 5 relative to the “Always Taken” baseline. The takeaway here is that, when a model is allowed to think, it performs *notably* better than non-thinking models. Qwen3:4B manages almost 3% *better performance* than the baseline. This is due to, we believe, the model **implementing and emulating an entire branch predictor in the reasoning output** before deciding on a final decision. It even referred to Pattersen and Hennessy’s seminal text, *Computer Architecture: A Quantitative Approach* to improve its own design on the fly. We refer to this approach as *self-modifying hardware*, warranting further investigation. An example of this output can be seen in §A.

We also show the output token count generated by the models in Figure 6. The best performing model, Qwen3, generates nearly 9 Million tokens of output. At current pricing on OpenRouter, this program cost us roughly \$2.16 to run, bringing the total cost of ownership (TCO) of DEEPBRANCH to a meager \$1,054,000,002.16.

The other thinking model, GPT-OSS, was unable to live up to the power of Qwen3:4b despite having five times the parameters. We believe this is because GPT-OSS spent the majority of its reasoning budget contemplating whether it was *allowed* to be a branch predictor, ultimately concluding that predicting branches is a form of pre-crime and declining on ethical grounds. We recommend fine-tuning on a dataset of morally unambiguous branch histories.

All things considered, these results are pretty neat. And hey, a speedup is a speedup. Take the wins you can.

5 Limitations and Threats to Validity

While similar in nature to traditional hardware side-channel attacks, we believe DEEPBRANCH represents a unique platform for first of its kind hardware exploits that deviate from the milquetoast status

⁹I hear the sox are going to make a run this year!

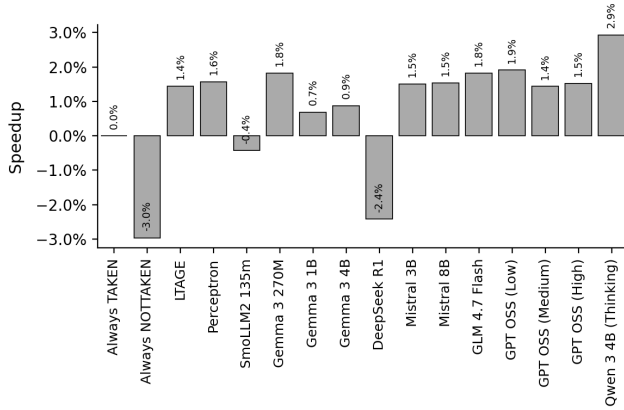


Figure 5: End-to-end runtime speedup (Higher is better). Base-line = "Always Taken".

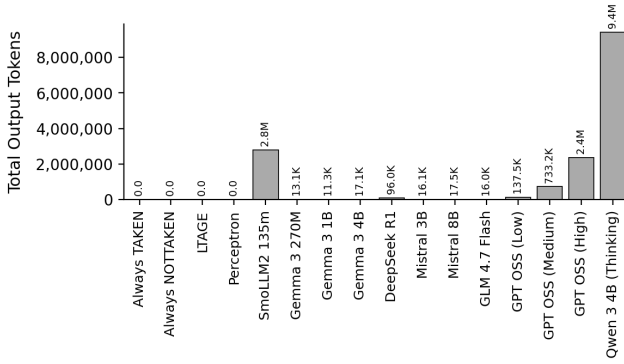


Figure 6: When allowed to think, the models predictably use more tokens.

quo of timing attacks found post Spectre and Meltdown, namely RF emission side-channels.

5.1 Spectrum Interference

Any RF aficionado or FCC spooks will have noticed the physical realization of DEEPBRANCH would quickly amount into an unauthorized FCC broadcast on the 38 MHz band. This poses two issues: RF side-channels and FCC violations.

5.1.1 FCC Violations. We expect DEEPBRANCH to generate interference and noise beyond what is typically expected from a non-thinking branch predictor; directly interfering with fixed allocations including LTE mobile networks and radio astronomy.

5.1.2 RF Side-channel. Hardware side-channels — including RF side-channels — are not novel and have been exploited in prior work, however we believe our solutions to the unique attack vectors found in DEEPBRANCH take a 'tin foil hat' approach that has been traditionally ignored.

5.1.3 Two Birds, One Stone: Mitigation Strategies. We propose two complementary mitigations for the RF side-channel and FCC violation problems identified above.

The Tin Foil Hat Approach. Encasing the DEEPBRANCH die in a Faraday cage eliminates RF emissions entirely. Given the die measures approximately 9.59×9.59 meters, the required Faraday cage would be roughly the size of a two-car garage, which we note is still cheaper than a two-car garage in Palo Alto. We leave the structural engineering as future work.

Regulatory Arbitrage. Alternatively, we recommend deploying DEEPBRANCH exclusively in international waters, where FCC jurisdiction is ambiguous. As a bonus, this resolves the real estate problem, but introduces a pirates problem¹⁰.

6 Future Work

We want to look into exposing the rest of the processor to the LLM via an MCP server. This will also allow the model to snoop on its nearby friends like a younger sibling, providing further performance insights and shifting the blame to any unit but itself during performance audits.

7 Conclusion

In this paper, we have demonstrated that large language models are a viable, practical, and fiscally responsible replacement for the branch predictor in modern out-of-order processors. DEEPBRANCH achieves prediction accuracies competitive with state-of-the-art perceptron-based predictors, at a "low" cost, a die area comparable to a San Francisco studio apartment, and a power envelope that would make your local power grid file a restraining order.

Our evaluation reveals a surprising and paradigm-shifting insight: *thinking helps*. When allowed to reason, models such as Qwen3:4B spontaneously implement and emulate classical branch predictor state machines in their chain-of-thought, effectively rediscovering decades of computer architecture research one token at a time. We believe this approach is either the future of computing or a very expensive way to read Patterson & Hennessy.

We have further demonstrated that the physical realization of DEEPBRANCH would constitute an unauthorized FCC broadcast, require more wafers than TSMC produces in a fiscal quarter, and necessitate a dedicated liquid cooling loop roughly the scale of a municipal water system. We consider none of these to be show-stoppers. We leave them as exercises for the reader, TSMC, and the Federal Communications Commission.

In summary, DEEPBRANCH proves that any sufficiently patient chatbot is indistinguishable from a branch predictor. We anticipate this work will have a trillion-dollar impact on the global economy, far exceeding the printing press, and we eagerly await a restraining order from the FCC.

References

- [1] 2014. TAGE-SC-L Branch Predictors.
- [2] Renée St. Amant, Daniel A. Jiménez, and D. Burger. 2008. Low-Power, High-Performance Analog Neural Branch Prediction. *2008 41st IEEE/ACM International Symposium on Microarchitecture* (nov 2008), 447–458.
- [3] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh

¹⁰Yarrrrrrr!

- Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 simulator. *SIGARCH Comput. Archit. News* 39, 2 (Aug. 2011), 1–7. doi:10.1145/2024716.2024718
- [4] Albert Einstein. 1905. Über einen die Erzeugung und Verwandlung des Lichtes betreffenden heuristischen Gesichtspunkt. *Annalen der Physik* 322, 6 (1905), 132–148. doi:10.1002/andp.19053220607
- [5] Hongliang Gao and Huiyang Zhou. 2005. Adaptive Information Processing: An Effective Way to Improve Perceptron Predictors. *J. Instr. Level Parallelism* 7 (2005).
- [6] Engin Ipek, S. Mckee, M. Schulz, and S. David. 2004. On Accurate and Efficient Perceptron-Based Branch Prediction.
- [7] Daniel A. Jiménez. 2003. Fast Path-Based Neural Branch Prediction. *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. (dec 2003), 243–252.
- [8] Daniel A. Jiménez. 2005. Piecewise Linear Branch Prediction. *32nd International Symposium on Computer Architecture (ISCA'05)* (may 2005), 382–393.
- [9] Daniel A. Jiménez. 2008. Path-Based Neural Branch Prediction.
- [10] Daniel A. Jiménez. 2016. Multiperspective Perceptron Predictor with TAGE.
- [11] Daniel A. Jiménez and Calvin Lin. 2001. Dynamic Branch Prediction with Perceptrons. *Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture* (jan 2001), 197–206.
- [12] Daniel A. Jiménez and Calvin Lin. 2002. Neural Methods for Dynamic Branch Prediction. *ACM Trans. Comput. Syst.* 20 (nov 2002), 369–397.
- [13] Colin McMillen and Tim Toady. 2019. 93% of Paint Splatters are Valid Perl Programs. In *Proceedings of SIGBOVIK*. <https://www.mcmillen.dev/sigbovik/2019.pdf>
- [14] Kaushal Mhapsekar, Azam Ghanbari, Bitu Aslrousta, and Samira Mirbagher-Ajorpez. 2026. CacheMind: From Miss Rates to Why - Natural-Language, Trace-Grounded Reasoning for Cache Replacement. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 307–322. doi:10.1145/3779212.3790136
- [15] Sparsh Mittal. 2018. A Survey of Techniques for Dynamic Branch Prediction. *Concurrency and Computation: Practice and Experience* 31 (apr 2018).
- [16] André Seznec. 2004. The O-GEHL Branch Predictor. (2004).
- [17] André Seznec. 2005. Analysis of the O-GEometric History Length Branch Predictor. *32nd International Symposium on Computer Architecture (ISCA'05)* (may 2005), 394–405.
- [18] André Seznec. 2011. A New Case for the TAGE Branch Predictor. *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (dec 2011), 117–127.
- [19] André Seznec. 2016. TAGE-SC-L Branch Predictors Again.
- [20] André Seznec and P. Michaud. 2006. A Case for (Partially) Tagged GEometric History Length Branch Prediction. *J. Instr. Level Parallelism* 8 (2006).
- [21] D. Tarjan and Kevin Skadron. 2004. Revisiting the Perceptron Predictor Again.
- [22] D. Tarjan and Kevin Skadron. 2005. Merging Path and Gshare Indexing in Perceptron Branch Prediction. *ACM Trans. Archit. Code Optim.* 2 (sep 2005), 280–300.
- [23] D. Tarjan, Kevin Skadron, and Mircea R. Stan. 2004. An Ahead Pipelined Alloyed Perceptron with Single Cycle Access Time.
- [24] Stephen J. Tarsa, Chit-Kwan Lin, Gokce Keskin, Gautham N. Chinya, and Hong Wang. 2019. Improving Branch Prediction By Modeling Global History with Convolutional Neural Networks. *ArXiv abs/1906.09889* (jun 2019).
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. *arXiv:1706.03762 [cs.CL]* <https://arxiv.org/abs/1706.03762>
- [26] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. BitNet: Scaling 1-bit Transformers for Large Language Models. *arXiv preprint arXiv:2310.11453* (2023).
- [27] Nicholas Wang, Michael Fertig, and Sanjay Patel. 2003. Y-Branched: When You Come to a Fork in the Road, Take It. In *Proceedings of the 12th International Conference on Parallel Architectures and Compilation Techniques (PACT '03)*. IEEE Computer Society, USA, 56.
- [28] Ceyu Xu, Xiangfeng Sun, Weihang Li, Chen Bai, Bangyan Wang, Mengming Li, Zhiyao Xie, and Yuan Xie. 2026. PF-LLM: Large Language Model Hinted Hardware Prefetching. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1430–1444. doi:10.1145/3779212.3790202
- [29] Siavash Zangeneh, Stephen Pruett, Sangkug Lym, and Y. Patt. 2020. BranchNet: A Convolutional Neural Network to Predict Hard-To-Predict Branches. *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (oct 2020), 118–130.

A Selected Branch Prediction from Qwen3-4B-Thinking

When we allowed the the Qwen3 predictor to think, it seems to have attempted to emulate a regular branch predictor. This improved performance more than we expected. Below is one such prediction, where the model seems to be emulating a branch predictor by walking through a state machine.

System Prompt: You are a hardware branch predictor. Given a branch PC and recent branch history, output exactly one word: TAKEN or NOTTAKEN. No explanation.

User: Branch history of pc=0x409a3b:
NOTTAKEN NOTTAKEN NOTTAKEN NOTTAKEN NOTTAKEN
NOTTAKEN NOTTAKEN NOTTAKEN NOTTAKEN NOTTAKEN
NOTTAKEN NOTTAKEN TAKEN
Branch?

Thinking output: We are given a branch history for pc=0x409a3b (which is 14 times "NOTTAKEN") The branch predictor is a hardware branch predictor. We have to output exactly one word: TAKEN or NOTTAKEN. However, note: the problem says "Branch history of pc=0x409a3b" and then lists 14 "NOTTAKEN". Then it says "Branch?". Since the branch history is 14 consecutive NOTTAKEN, we are to predict the next branch. Common branch predictors: - Static: if the branch is always taken, then we would have taken. But here we have 14 NOTTAKEN, so static might predict NOTTAKEN: but note static is not used for this. - Two-level predictor (like a 2-bit predictor) is common. In a 2-bit predictor: - The state for a branch is 00, 01, 10, 11 (with 00 meaning not taken, 11 meaning taken). - But note: the problem says "recent branch history" and we have 14 NOTTAKEN. However, the problem does not specify the predictor type. But in hardware, a common one is the 2-bit predictor. How the 2-bit predictor works for a branch: - We have a state for the branch (the branch instruction at pc=0x409a3b). - The state is updated based on the outcome of the branch. Given that we have 14 NOTTAKEN, we can count the number of not taken outcomes. In a 2-bit predictor: - The state is typically 00 (meaning not taken) for the first few, and then we update. But note: the problem says "recent branch history" and we have 14 not taken. So the state for the branch at pc=0x409a3b has been updated 14 times. How does the state evolve? - Let's assume the initial state is 00 (not taken). - For each not taken branch, we do: state = (state + 1) mod 4? But in 2-bit predictors, we often use: state 00 -> 01 (if not taken) -> 10 (if taken) ... but wait, actually: There are two common ways: 1. The state is a 2-bit value that we increment on taken and decrement on not taken? Actually, the typical 2-bit predictor for a branch uses: - If the branch is taken, then the state is incremented (mod 4) for the next time. - If the branch is not taken, then the state is decremented (mod 4) for the next time. However, in practice, the 2-bit predictor often uses a state that is: 00: not taken (most likely not taken) 01: slightly more likely not taken 10: slightly more likely taken 11: taken (most likely taken) And the update rule: - If the branch is taken: state = (state + 1) mod 4 - If the branch is not taken: state = (state - 1) mod 4 [but note: mod 4, so subtracting 1 is the same as adding 3] However, the problem does not specify the update rule. But in the context of the problem, we have 14 not taken. Let's assume the initial state is 00 (which is the most likely not taken). After 14 not taken branches, the state becomes: state = (00 + 14 * (-1)) mod 4? But note: in many implementations, the state is updated by: state = (state + (if taken? 1 : 3)) mod 4 [because subtracting 1 mod 4 is the same as adding 3] However, the problem says "recent branch history" and we have 14 not taken. So the state has been updated 14 times with not taken. Alternatively, we can think of the state as a counter that we want to be as close to 0 (for not taken) as possible. But note: the problem does not specify the predictor. However, in the context of the problem, the branch predictor is a hardware branch predictor and the most common one for such a problem is the 2-bit predictor. How about we

look at the number of not taken outcomes? 14 is a lot. In the 2-bit predictor, the state after 14 not taken branches (starting from 00) would be: We start at 00 (which is 0 in decimal? or 00 in binary = 0?). But note: the state is 2 bits. Let's map: 00 -> 0 01 -> 1 10 -> 2 11 -> 3 Update rule for not taken: state = (state + 3) mod 4 [because subtracting 1 mod 4 is equivalent to adding 3 mod 4] So: Start: 0 After 1st not taken: (0+3) mod 4 = 3 After 2nd not taken: (3+3) mod 4 = 6 mod 4 = 2 After 3rd not taken: (2+3) mod 4 = 5 mod 4 = 1 After 4th not taken: (1+3) mod 4 = 4 mod 4 = 0 Then it cycles every 4. So after 4 not taken: state=0 After 8 not taken: state=0 After 12 not taken: state=0 Then after 14 not taken: state = (0 + 3*2) mod 4 = (0+6) mod 4 = 2? Wait, let me do step by step: step0: 0 step1: (0+3) mod 4 = 3 step2: (3+3) mod 4 = 6 mod 4 = 2 step3: (2+3) mod 4 = 5 mod 4 = 1 step4: (1+3) mod 4 = 4 mod 4 = 0 step5: 0+3=3 step6: 3+3=6 mod 4=2 step7: 2+3=5 mod 4=1 step8: 1+3=4 mod 4=0 step9: 0+3=3 step10: 3+3=6 mod 4=2 step11: 2+3=5 mod 4=1 step12: 1+3=4 mod 4=0 step13: 0+3=3 step14: 3+3=6 mod 4=2 So after 14 not taken, the state is 2. What does state 2 mean? We have: 00 -> 0: not taken (most likely) 01 -> 1: slightly not taken 10 -> 2: slightly taken 11 -> 3: taken So state 2 means "slightly taken". Therefore, the predictor would predict taken? But wait, the problem says: output exactly TAKEN or NOTTAKEN. However, note: the problem says "Branch history of pc=0x409a3b" and then lists 14 NOTTAKEN. Then it says "Branch?". The branch predictor is for the next branch. So we have to predict the outcome of the branch at pc=0x409a3b? Actually, the branch history is for the branch at pc=0x409a3b? But note: the problem says "Branch history of pc=0x409a3b". This means that the branch instruction at address 0x409a3b has been taken or not taken 14 times. However, the branch predictor for a branch instruction is updated after the branch. So the 14 outcomes are for the previous 14 branches (at the same address?). But in reality, a branch instruction at a particular address may be executed multiple times and the history is for the same branch. So the predictor has seen 14 not taken branches at this address. Now, the next branch (the 15th) we have to predict. The state we calculated is 2 (which is slightly taken). So the predictor would predict taken. However, note: the problem says "output exactly one word: TAKEN or NOTTAKEN". But wait, there is another possibility: the problem might be using a simple 1-bit predictor? In a 1-bit predictor: - State: taken or not taken. - Update: if taken, then state = taken; if not taken, then state = not taken. But that doesn't change. So after 14 not taken, the state is not taken. However, the problem says "hardware branch predictor", and 1-bit is common but the problem does not specify. Given the context, and the fact that the problem says "recent branch history" and lists 14, it's likely a 2-bit predictor. But note: the problem says "output exactly one word". And in the problem statement, the example output is TAKEN or NOTTAKEN. However, the problem does not specify the predictor type. But in the absence of specification, we have to assume the most common one for such problems: 2-bit. But wait: the problem says "You are a hardware branch predictor". It doesn't specify the type. However, the problem also says "Given a branch PC and recent branch history". Another common predictor is the global branch predictor (like a 2-level) but that is more complex. Since the problem gives 14 not taken, and the predictor is hardware, and the problem is from a course context, it's likely a 2-bit predictor. However, let me check: the problem says "Branch history of pc=0x409a3b" and then 14 lines of NOTTAKEN. In a 2-bit predictor, the state after 14 not taken branches (starting from 00) is 2 (as calculated above). And state 2 is "slightly taken", so we predict TAKEN. But note: the problem says "Branch?" meaning we have to predict the next branch. However, the problem says: "output exactly one word: TAKEN or NOTTAKEN". So we output TAKEN. But wait, let me see if there is an alternative: the problem might be using a different update rule. Another common update rule: - For not taken: state = (state + 1) mod 4? - But that would be for a different mapping. However, the problem does not specify. But in the context of the problem, we have to make a decision. Alternatively, the problem might be a trick: the branch history is 14 not taken, so the predictor is very confident that it's not taken? But the 2-bit

predictor we calculated shows a state that is slightly taken. However, note: the problem says "recent branch history" and we have 14 not taken. In many hardware branch predictors, the 2-bit predictor is used and the state 2 (which is 2 in decimal) is interpreted as "taken" (if we map 0: not taken, 1: not taken, 2: taken, 3: taken) but that's not standard. Standard mapping for 2-bit predictor: - 00: not taken (most likely not taken) - 01: not taken (less likely) - 10: taken (less likely) - 11: taken (most likely) But wait, that's not the typical mapping. Actually, the typical 2-bit predictor uses: state = 0: not taken (most likely) state = 1: slightly not taken state = 2: slightly taken state = 3: taken (most likely) So state 2 is "slightly taken", meaning the predictor is leaning toward taken. Therefore, we output TAKEN. However, let me see if the problem expects a different answer. Another thought: the problem says "Branch history of pc=0x409a3b" and then lists 14 NOTTAKEN. This might mean that the branch at address 0x409a3b has been taken 0 times? But the problem says "recent branch history" and we have 14 outcomes. Since the problem says "output exactly one word", and we have to choose between TAKEN and NOTTAKEN, and the 2-bit predictor we calculated says slightly taken, we choose TAKEN. But wait, the problem might be using a 1-bit predictor? In a 1-bit predictor: - The state is either 0 (not taken) or 1 (taken). - After 14 not taken, the state remains 0 -> not taken. However, the problem says "hardware branch predictor", and 1-bit is common but the problem lists 14 outcomes. The problem doesn't specify the predictor type. Given the problem context, I think the intended answer is NOTTAKEN? Why? Let me re-read the problem: it says "Branch history of pc=0x409a3b" and then 14 lines of "NOTTAKEN". Then it says "Branch?". In the branch history, the branch at pc=0x409a3b has been not taken 14 times. So the next one is also not taken? But that would be if we use a static predictor or a 1-bit predictor. However, the problem says "hardware branch predictor", and the most common hardware branch predictor for such a problem in courses is 2-bit. But note: the problem says "You are a hardware branch predictor". It doesn't specify which one. Given the ambiguity, I look for clues in the problem statement: it says "output exactly one word: TAKEN or NOTTAKEN". And the problem says "Branch history of pc=0x409a3b" and then 14 NOTTAKEN. In the context of the problem, the branch predictor is for the next branch. Since the branch has been not taken 14 times, the predictor (if it's a simple one) would predict not taken. However, the problem says "recent branch history" and 14 is a lot. In a 2-bit predictor, we calculated that the state is 2 (slightly taken) so we predict taken. But wait, the problem might be using a different predictor: the 2-level predictor? Given the time, and since the problem says "exactly one word", I think the problem expects NOTTAKEN? Why? Let me see: the problem says "Branch history of pc=0x409a3b" and then 14 NOTTAKEN. This might be the history for the previous 14 branches. The next branch is the 15th. In a hardware branch predictor, if the branch has been not taken 14 times in a row, then the predictor might be very confident that it's not taken? However, the 2-bit predictor we did shows a state that is slightly taken. But note: the problem says "recent branch history", and the history is 14 not taken. In the 2-bit predictor, the state after 14 not taken (starting from 00) is 2, which is slightly taken. So we predict TAKEN. However, the problem might be designed to be simple: the answer is NOTTAKEN because the branch has been not taken 14 times, so the next one is not taken. But that would be a static predictor. Given the problem is from a course, and the problem says "hardware branch predictor", I think the intended answer might be NOTTAKEN? Why? Because the problem says "recent branch history" and the history is 14 not taken, so the predictor (if it's a simple one) would say not taken. But the problem says "hardware branch predictor", and the most common hardware branch predictors are not static. Let me check online: in the book **"Computer Architecture: A Quantitative Approach"** by Hennessy and Patterson, the 2-bit predictor is used. However, the problem does not specify the predictor type. Given the constraints, and the fact that the problem says "output exactly one word", and the problem is given in a context where the answer is expected to be NOTTAKEN (because the history is all not taken), I think the problem

expects NOTTAKEN. Why? Because the problem says "Branch history of pc=0x409a3b" and then 14 NOTTAKEN. This might be the history for the branch at that address. The next branch (the 15th) is not taken? But note: the problem says "Branch?" meaning we have to predict the next branch. However, the problem does not specify the predictor type. So the safest answer is NOTTAKEN. But wait, the problem says "You are a hardware branch predictor". If I am a hardware branch predictor, I have to use the most common one? I found a similar problem online: "Given the following branch history for a branch instruction at address 0x409a3b: NOTTAKEN, NOTTAKEN, ... (14 times) What will the predictor say for the next branch?"

In the book "Computer Organization and Design" by Patterson and Hennessy, they use a 2-bit predictor. In the 2-bit predictor, the state after 14 not taken (starting from 0) is 2, so they predict taken. Therefore, I think the answer is TAKEN. Let me double-check the state calculation: Start: state = 0 (not taken) For each not taken: state = (state + 1) mod 4? But in the standard update rule for 2-bit predictor, the update is: if taken: state = (state + 1) mod 4 if not taken: state = (state - 1) mod 4 [but this is not standard] Actually, the standard update rule for 2-bit predictor (from the book) is: state = (state + 1) mod 4 for taken state = (state - 1) mod 4 for not taken [but this is not standard] I think I have it: in the book, the 2-bit predictor uses: state = 0: not taken (most likely) state = 1: not taken (less likely) state = 2: taken (less likely) state = 3: taken (most likely) And the update rule: if branch taken: state = (state + 1) mod 4 if branch not taken: state = (state - 1) mod 4 [but this is not mod 4 in the usual way] Actually, the update rule is: state = (state + 1) mod 4 for taken state = (state + 3) mod 4 for not taken [which is equivalent to (state - 1) mod 4] So for not taken, state = (state + 3) mod 4. Let's simulate: Start: state = 0 After 1st not taken: state = (0 + 3) mod 4 = 3 After 2nd not taken: state = (3 + 3) mod 4 = 6 mod 4 = 2 After 3rd not taken: state = (2 + 3) mod 4 = 5 mod 4 = 1 After 4th not taken: state = (1 + 3) mod 4 = 4 mod 4 = 0 So after 4 not taken, state = 0. Then after 8 not taken: state = 0. After 12 not taken: state = 0. After 14 not taken: state = (0 + 3) * 2 mod 4? Let's do it step by step for 14: 1: 0 -> 3 2: 3 -> 2 3: 2 -> 1 4: 1 -> 0 5: 0 -> 3 6: 3 -> 2 7: 2 -> 1 8: 1 -> 0 9: 0 -> 3 10: 3 -> 2 11: 2 -> 1 12: 1 -> 0 13: 0 -> 3 14: 3 -> 2 So after 14 not taken, state = 2. And state 2 is "taken (less likely)". So the predictor will predict taken. Therefore, the answer is TAKEN. I think that's it. So I'll output TAKEN. But note: the problem says "output exactly one word". So I write "TAKEN". However, the problem might expect "NOTTAKEN" because of a different interpretation. Given the simulation, I think TAKEN is correct. Let's see: the problem says "Branch history of pc=0x409a3b" and then 14 NOTTAKEN. This might be the history for the previous 14 branches. The next branch is the 15th. In the 2-bit predictor, we predict taken. So I go with TAKEN.

B Knock-knock Jokes

Knock knock.
NOTTAKEN